



Nome: _____

Grupo: _____

Nome: _____

Grupo: _____

Exercício 1 – Cadastro de Equipamentos

Em Java, faça um sistema simples de cadastro de equipamentos. A aplicação deve ser capaz de gerenciar dois tipos de equipamentos: Equipamento e Computador. A classe Equipamento deve conter os seguintes atributos: nome (do tipo String) e modelo (do tipo String). Estes atributos representam as características gerais de qualquer equipamento.

Crie um construtor na classe Equipamento que inicialize esses atributos e também um método `exibirDados()`, que deverá imprimir na tela o nome e o modelo do equipamento.

Além disso, crie uma classe chamada Computador, que herda da classe Equipamento, ou seja, Computador é um tipo especializado de Equipamento. A classe Computador deve adicionar um novo atributo chamado `processador` (do tipo String), que será específico para computadores.

Na classe Computador, o construtor deve receber o nome, o modelo e o processador, utilizando o método `super()` para inicializar os atributos nome e modelo da classe pai (Equipamento). A classe Computador deve sobrescrever o método `exibirDados()`, para além do nome e modelo, também exibir o processador do computador.

Por fim, crie um arquivo de teste onde você instanciará um objeto da classe Computador, passando valores para os atributos nome, modelo e processador, e chamará o método `exibirDados()` para garantir que as informações estão sendo exibidas corretamente.

Exercício 2 – Sistema de Pagamentos

Em Java, faça um sistema simples para gerenciar o processamento de pagamentos. O sistema será composto por uma classe abstrata chamada Pagamento, que servirá como base para diferentes tipos de pagamento. A classe Pagamento terá

dois atributos principais: valor (do tipo double), que representará o valor do pagamento, e status (do tipo StatusPagamento), que indicará o status atual do pagamento. O status será inicializado com o valor PENDENTE e o valor será fornecido através do construtor da classe. Além disso, a classe Pagamento incluirá um método abstrato chamado processarPagamento(), que será implementado pelas subclasses para definir como o pagamento será processado. Também haverá um método exibirStatus(), que imprime o status atual do pagamento.

Em seguida, crie uma enumeração chamada StatusPagamento para representar os diferentes estados possíveis de um pagamento: PENDENTE, PROCESSANDO, CONCLUIDO e FALHA.

A partir da classe Pagamento, crie uma subclass chamada PagamentoCartao. Esta classe deve adicionar um novo atributo chamado numeroCartao (do tipo String), que armazenará o número do cartão utilizado para o pagamento. O construtor de PagamentoCartao deve inicializar o valor e o número do cartão, além de chamar o construtor da classe Pagamento para inicializar o status do pagamento. A classe PagamentoCartao deve sobrescrever o método processarPagamento(), onde será simulada a conclusão do pagamento e o status será alterado para CONCLUIDO. O método também exibirá os últimos 4 dígitos do número do cartão ao processar o pagamento.

Por fim, crie um arquivo de teste onde você instanciará um objeto da classe PagamentoCartao, passando valores para o valor do pagamento e o número do cartão. Chame o método exibirStatus() para verificar o status do pagamento antes e depois do processamento. O objetivo é garantir que o sistema altere corretamente o status do pagamento de PENDENTE para CONCLUIDO após o processamento.

Exercício 3 – Sistema de Usuários

Crie um sistema em Java para um sistema de controle de usuários utilizando o conceito de encapsulamento e enumeração. A primeira parte do exercício consiste em criar um enum chamado NivelAcesso, que representa os diferentes níveis de acesso de um usuário no sistema: ADMIN, USUARIO e CONVIDADO. Este enum será utilizado para definir o nível de acesso dos usuários.

A classe Usuario deve ser criada com os seguintes atributos:

- login: uma variável do tipo String que armazenará o nome de usuário.
- senha: uma variável do tipo String que armazenará a senha do usuário.

- nivel: uma variável do tipo NivelAcesso que indicará o nível de acesso do usuário, sendo um dos valores do enum NivelAcesso.
- ativo: uma variável do tipo boolean que indica se o usuário está ativo ou não, sendo inicializada com o valor true.

A classe Usuario deve fornecer um construtor que inicializa os atributos login, senha, nivel e ativo. Além disso, deve ter os seguintes métodos:

- autenticar(String senha): este método deve verificar se a senha informada é válida e se o usuário está ativo. Se ambas as condições forem verdadeiras, o método retorna true; caso contrário, retorna false.
- desativar(): este método desativa o usuário, alterando o valor do atributo ativo para false.
- getNivel(): retorna o nível de acesso do usuário.
- getLogin(): retorna o nome de login do usuário.
- Atenção: Não é necessário criar um método getSenha() por questões de segurança. Senhas nunca devem ser expostas diretamente.

Em seguida, no arquivo de teste, crie um objeto da classe Usuario representando um administrador (ADMIN). No teste, verifique se a autenticação do usuário funciona corretamente, chamando o método autenticar com a senha correta. Além disso, imprima o nível de acesso do usuário utilizando o método getNivel().

Exercício 4 – Sistema de Notificações

Com a linguagem de programação Java crie um sistema simples de notificações utilizando herança, polimorfismo e enums. A classe Notificacao será a classe base, com atributos comuns como destinatario, mensagem e tipo (usando o enum TipoNotificacao para definir os tipos: EMAIL, SMS, PUSH).

As subclasses Email e SMS irão sobrescrever o método enviar() para implementar o envio específico de cada tipo de notificação. A classe SMS também fará o truncamento da mensagem para garantir que o texto não ultrapasse 160 caracteres.

No teste, será criado um array de notificações, contendo tanto Email quanto SMS, e, para cada notificação, os métodos preparar() e enviar() serão chamados para demonstrar como o sistema funciona.

O objetivo é entender os conceitos de herança, polimorfismo e como essas técnicas podem ser aplicadas no envio de notificações em Java.

Exercício 5 – Sistema simples de zoológico (Exercício adicional)

Utilizando a linguagem Java desenvolva um sistema simples de zoológico em Java, aplicando os principais conceitos de Programação Orientada a Objetos (POO) como abstração, herança, polimorfismo, encapsulamento e o uso de enum.

O sistema é baseado na criação de uma classe abstrata `Animal`, que representa um animal genérico com atributos encapsulados como nome, idade e habitat (este último representado por um enum `Habitat`, com valores `TERRESTRE`, `AQUATICO` e `AEREO`). A classe define um método abstrato `emitirSom()` e um método concreto `exibirInfo()` para exibir informações do animal.

Três subclasses herdam de `Animal`: `Leao`, `Golfinho` e `Arara`. Cada uma representa um tipo específico de animal e implementa de forma diferente o método `emitirSom()`, demonstrando o uso de **polimorfismo**.

No teste, é criada a classe `TesteZoologico`, onde instâncias de `Leao`, `Golfinho` e `Arara` são criadas e testadas. Para cada uma, os métodos `exibirInfo()` e `emitirSom()` são chamados, validando o comportamento das subclasses